

Fundamentos de Deep Learning & Neurônio Artificial

Aula 01: O Perceptron, Ativações e o Caso da Creatina

Faculdade Infnet

Redes Neurais Profundas (Deep Learning e Visão Computacional)

Data: 2026-07-27

Apresentação do Professor

Conheça quem estará com você nesta jornada



Allan Spadini

Professor & Pesquisador em IA

DOUTORADO

Doutor em Geofísica

Sólida formação quantitativa, modelagem física e matemática aplicada.

ESPECIALIZAÇÃO

Pós em Ciência de Dados

Especialista em análise estatística, aprendizado de máquina e dados massivos.

ATUAÇÃO

Pesquisador em Inteligência Artificial

Desenvolvimento e pesquisa em agentes inteligentes.

Roteiro & Objetivos da Aula

Quatro blocos essenciais para dominar a fundação do Deep Learning



ETAPA 01

Perceptron & Decisão Linear

Entradas, Pesos, Viés e a Geometria do espaço cartesiano.

ETAPA 02

Funções de Ativação

Sigmoid, Tanh, ReLU, Leaky ReLU e o combate ao desvanecimento do gradiente.

ETAPA 03

Limitações & MLPs

O histórico problema do XOR e a solução através de camadas ocultas.

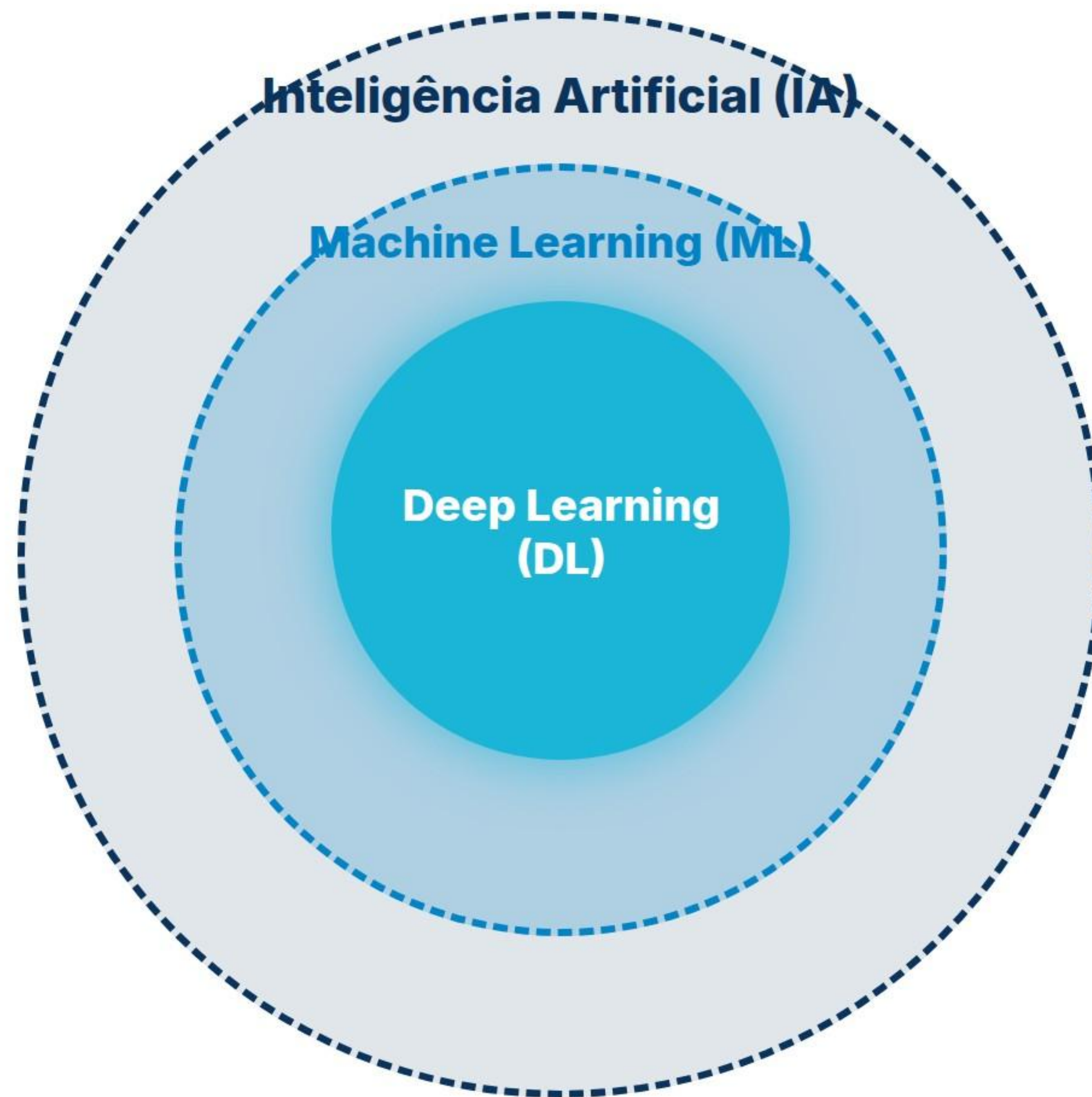
ETAPA 04

Planilha & Transição PyTorch

Ajuste manual na planilha excel e introdução às abstrações do PyTorch.

O Universo da Inteligência Artificial

Entendendo a hierarquia: IA vs Machine Learning vs Deep Learning



💡 Clique em qualquer círculo para detalhar o conceito!

MÓDULO SELECIONADO

Deep Learning (DL)

Subcampo do ML baseado em Redes Neurais Profundas com múltiplas camadas que aprendem hierarquias de representação.

Exemplos Principais:

Perceptrons & MLPs

Redes Convolucionais (CNNs)

Transformers (GPT, BERT)

Autoencoders

ML Clássico vs. Deep Learning

A revolução do aprendizado automático de representações



MACHINE LEARNING CLÁSSICO

Feature Engineering Manual

- ▶ Engenheiros extraem características manualmente.
- ▶ Depende fortemente do conhecimento de domínio.
- ▶ Algoritmos simples (SVM, Random Forest, Regressão).
- ▶ Desempenho estagna em grandes volumes de dados.

DEEP LEARNING

Aprendizado Fim-a-Fim (End-to-End)

- ▶ Dados brutos (pixels, texto, áudio) entram diretamente.
- ▶ Camadas ocultas aprendem representações abstratas.
- ▶ Escala massiva com grandes quantidades de dados e GPUs.
- ▶ Elimina a necessidade de extração manual de atributos.

A Origem Biológica do Perceptron

Clique nas partes do neurônio para ver a analogia entre biologia e matemática

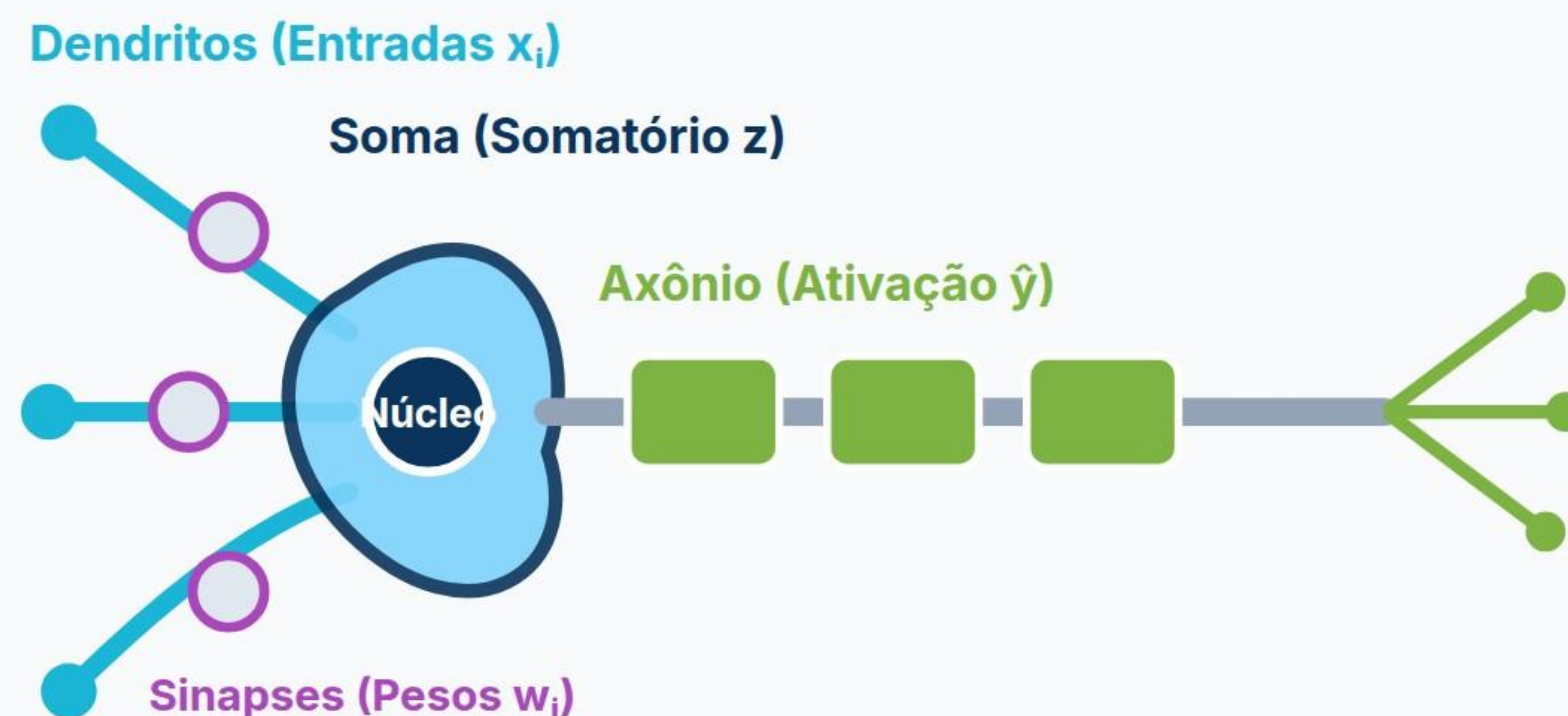


 Neurônio Biológico Humano

 Perceptron Artificial

 Clique nas partes do neurônio para ver a analogia!

Anatomia do Neurônio Humano Biológico



Dendritos

Sinapses

Soma

Axônio

 **BIOLÓGICO: Dendritos (Recepção de Sinais)**

 **Entradas de Dados ($x_1, x_2, \dots, x_n$)**

Estrutura Biológica: Ramificações celulares que captam neurotransmissores e impulsos elétricos vindos de outros neurônios.

 **Analogia Matemática:** Correspondem aos atributos brutos de entrada (ex: dosagem de creatina, horas de treino).

Fluxo de Processamento de um Neurônio

As duas etapas internas essenciais de cálculo



1. Soma Ponderada (z)

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Combinação linear das entradas multiplicadas pelos pesos mais o viés.

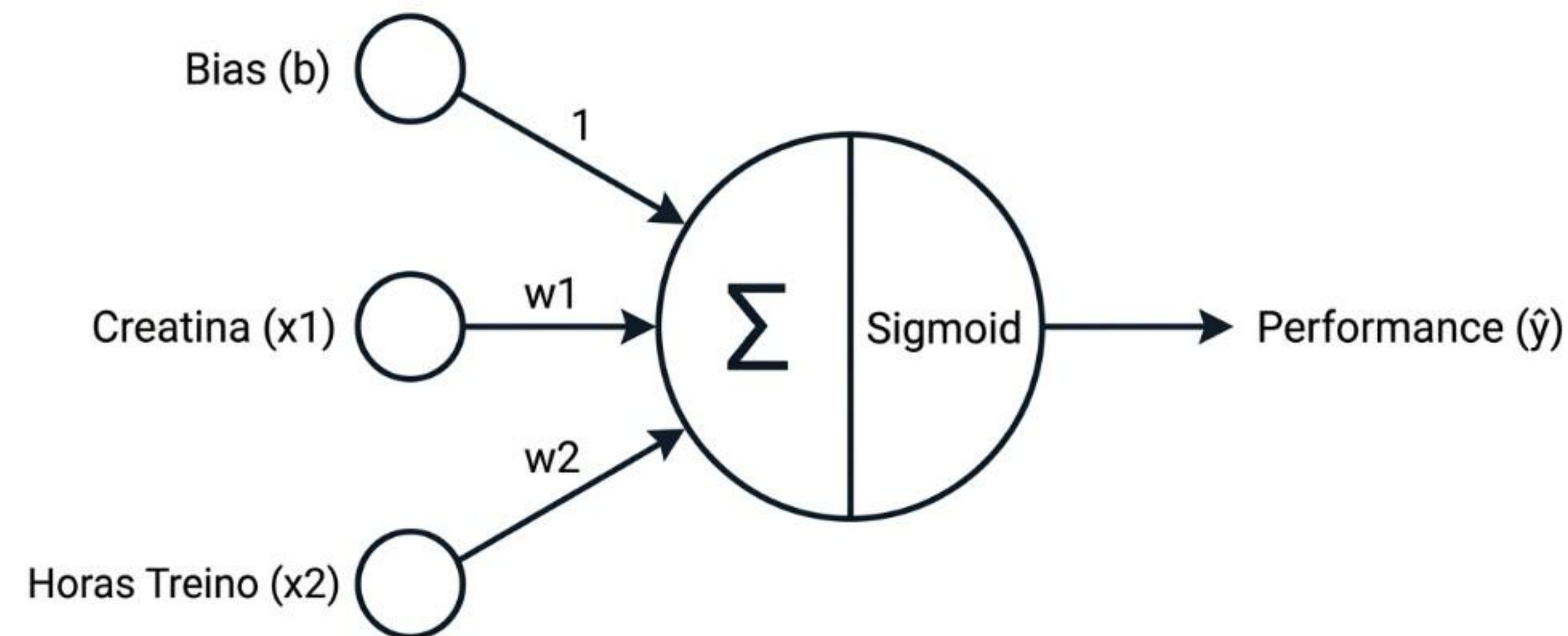
2. Ativação ($\hat{y}$)

$$\hat{y} = f(z)$$

Aplicação de uma função $f(z)$ para decidir o nível de saída do neurônio.

Caso Prático: O Problema da Creatina

Classificando atletas de Alta Performance vs Regular



- ▶ Entrada x_1 : Consumo diário de Creatina (em gramas).
- ▶ Entrada x_2 : Horas de Treino por semana.
- ▶ Saída y : Performance (1 = Alta Performance, 0 = Regular).
- ▶ Objetivo: Encontrar os pesos w_1 , w_2 e o viés b que separem os atletas.

A Equação da Decisão Linear

Entendendo a geometria do Potencial Linear (z)



$$z = w_1x_1 + w_2x_2 + b$$

z (Potencial)

Valor numérico sem limite. Se $z > 0$, o neurônio tende à classe 1.

w_1, w_2 (Pesos)

Determinam a inclinação da reta no gráfico cartesiano.

b (Viés / Bias)

Permite deslocar a reta. Sem "b", a reta passaria obrigatoriamente pela origem (0,0).

Simulador Interativo do Perceptron

Ajuste os sliders de w_1 , w_2 e b para encontrar a reta divisória perfeita!



⚙️ Parâmetros do Neurônio

Acurácia: 100%

Peso W_1 (Creatina): 1.20



Peso W_2 (Horas Treino): 0.90



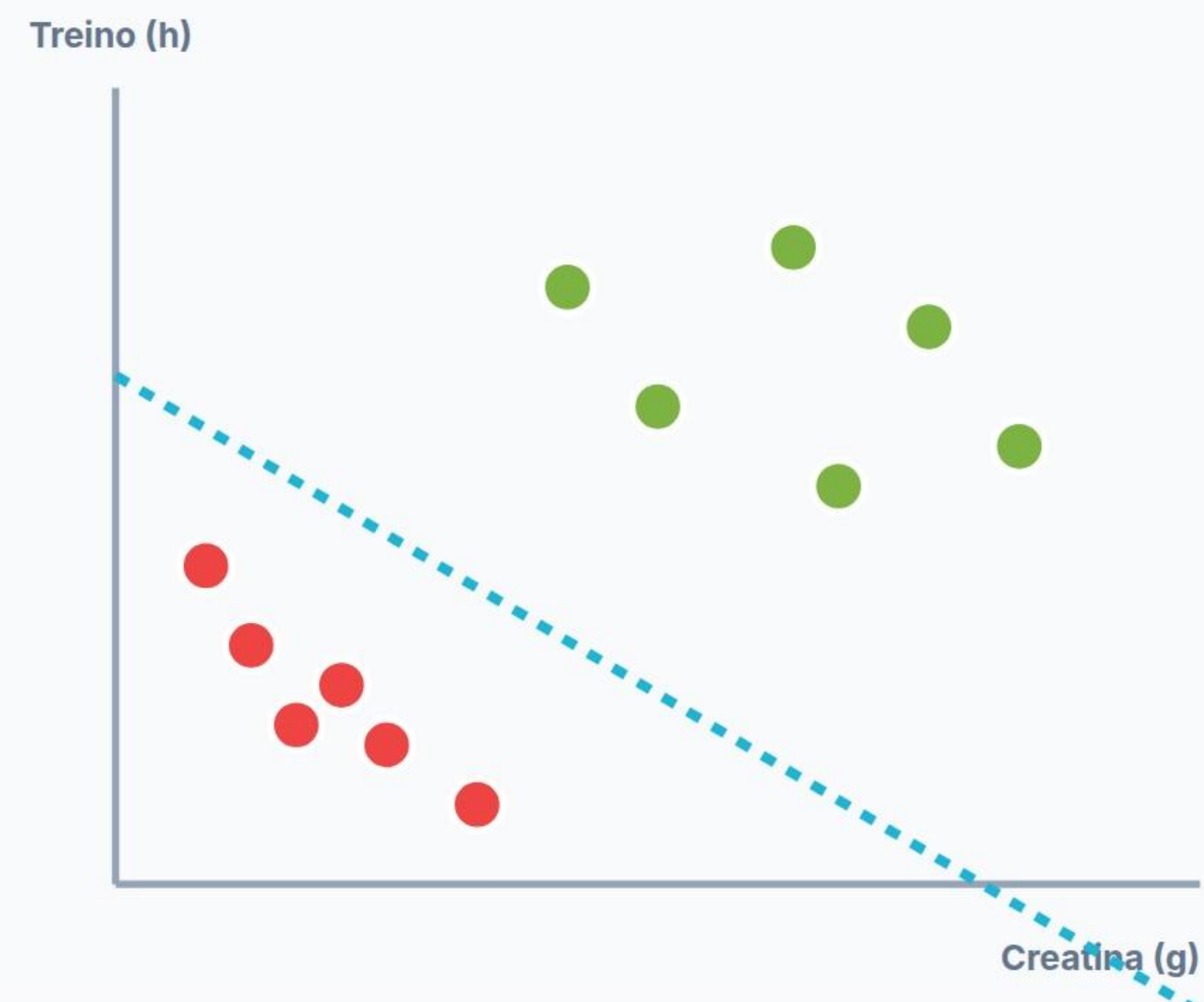
Viés (Bias - b): -11.5



$$z = (1.2 \times x_1) + (0.9 \times x_2) + (-11.5)$$

↺ Resetar para Solução Ótima

Fronteira de Decisão: $1.2 \cdot x_1 + 0.9 \cdot x_2 + (-11.5) = 0$



● Alta Performance (1) ● Regular (0) -- Reta $z = 0$

Necessidade de Probabilidades: A Função Sigmoid

Convertendo a soma ponderada z para o intervalo de probabilidade $[0, 1]$



$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Intervalo $[0, 1]$

Mapeia qualquer valor z entre $-\infty$ e $+\infty$ para um valor entre 0.0 (0%) e 1.0 (100%).

Regra de Decisão

Se $\hat{y} \geq 0.5$ (50%), classificamos como Alta Performance (1). Caso contrário, Regular (0).

Fronteira em $z = 0$

Quando $z = 0$, a Sigmoid retorna exatamente $\sigma(0) = 0.5$.

Plotter Interativo da Sigmoid

Arraste o slider de z e observe o achatamento da curva em S



Sigmoid

Tanh

ReLU

Leaky

Entrada (z):

1.50

Saída f(z):
0.8176

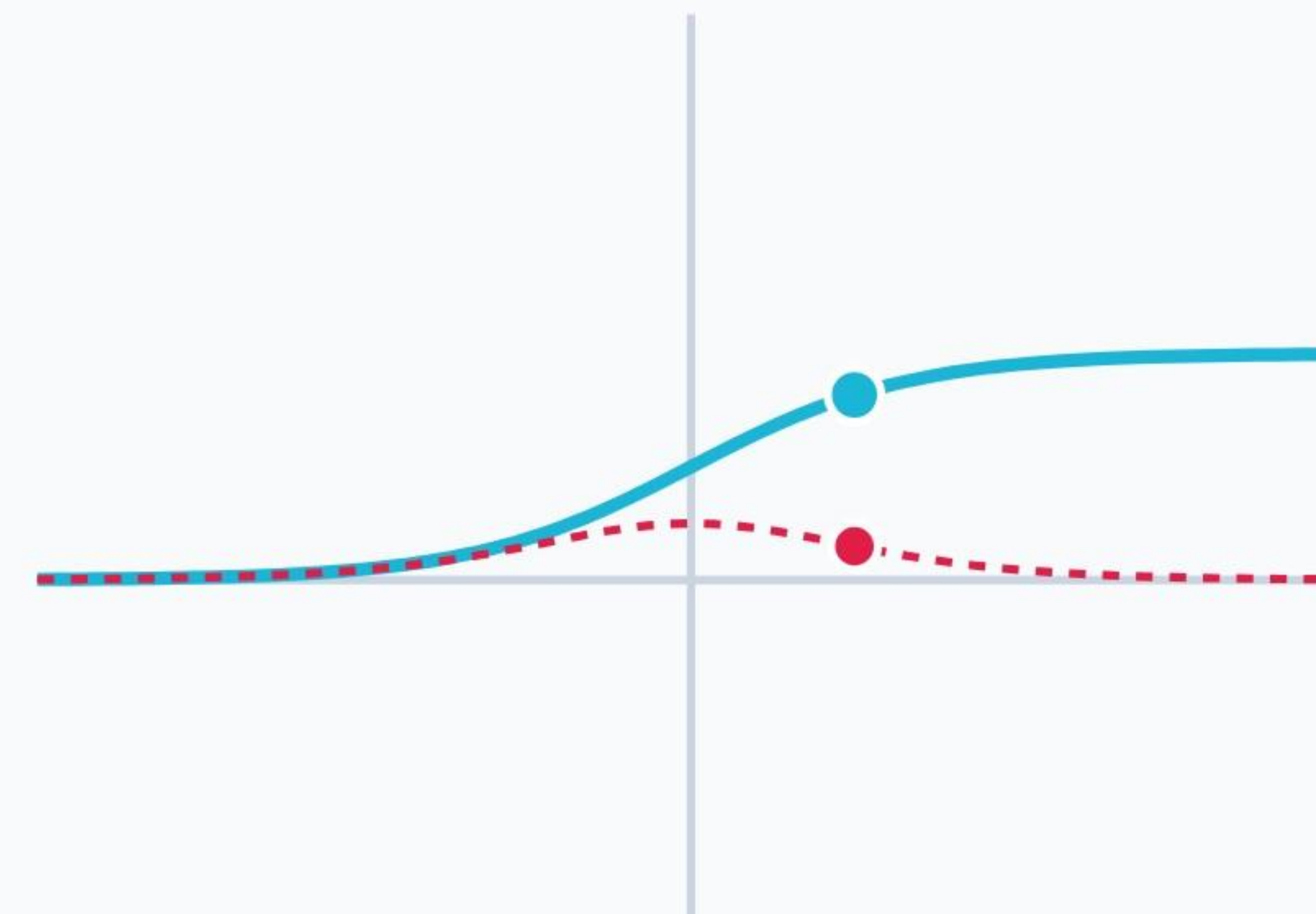
Gradiente f'(z):
0.1491

Sigmoid (Intervalo: [0, 1])

Achata qualquer entrada para [0, 1]. Excelente para probabilidades no final de classificações binárias.

— f(z)

- - Derivada f'(z)



$$\sigma(z) = 1 / (1 + e^{-z})$$

Por Que Precisamos de Não-Linearidades?

O colapso matemático de redes profundas sem funções de ativação



$$y = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W_{eq}x + b_{eq}$$

Colapso Linear

Multiplicar matrizes lineares produz apenas outra matriz linear equivalente.

Rede de 100 Camadas

Sem funções de ativação não-lineares, uma rede de 100 camadas é matematicamente idêntica a 1 único neurônio!

Poder de Representação

Ativações atuam como "dobradiças" que curvam o espaço cartesiano.

Visão Geral das Funções de Ativação

Comparativo dos 4 modelos fundamentais em gráficos vetoriais de alta resolução



PROBABILIDADE BINÁRIA

Intervalo: $[0, 1]$

Sigmoid

$$\sigma(z) = 1 / (1 + e^{-z})$$

Usada na camada final de classificação binária.



CENTRADOS EM ZERO

Intervalo: $[-1, 1]$

Tanh (Tangente Hiperbólica)

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

Facilita a convergência nas camadas ocultas intermediárias.



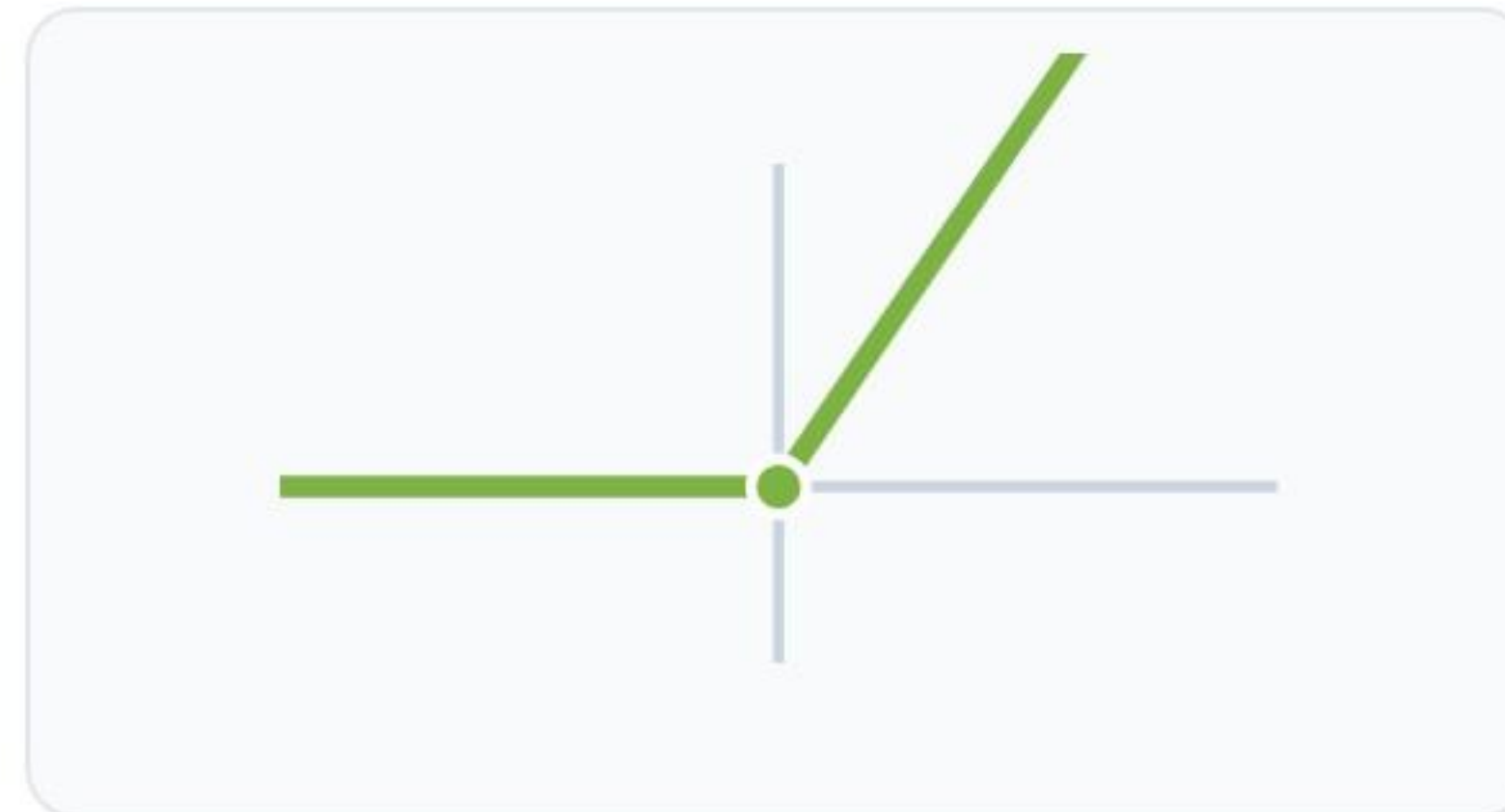
PADRÃO OURO DEEP LEARNING

Intervalo: $[0, +\infty)$

ReLU

$$f(z) = \max(0, z)$$

Super rápida em GPUs e evita o Vanishing Gradient.



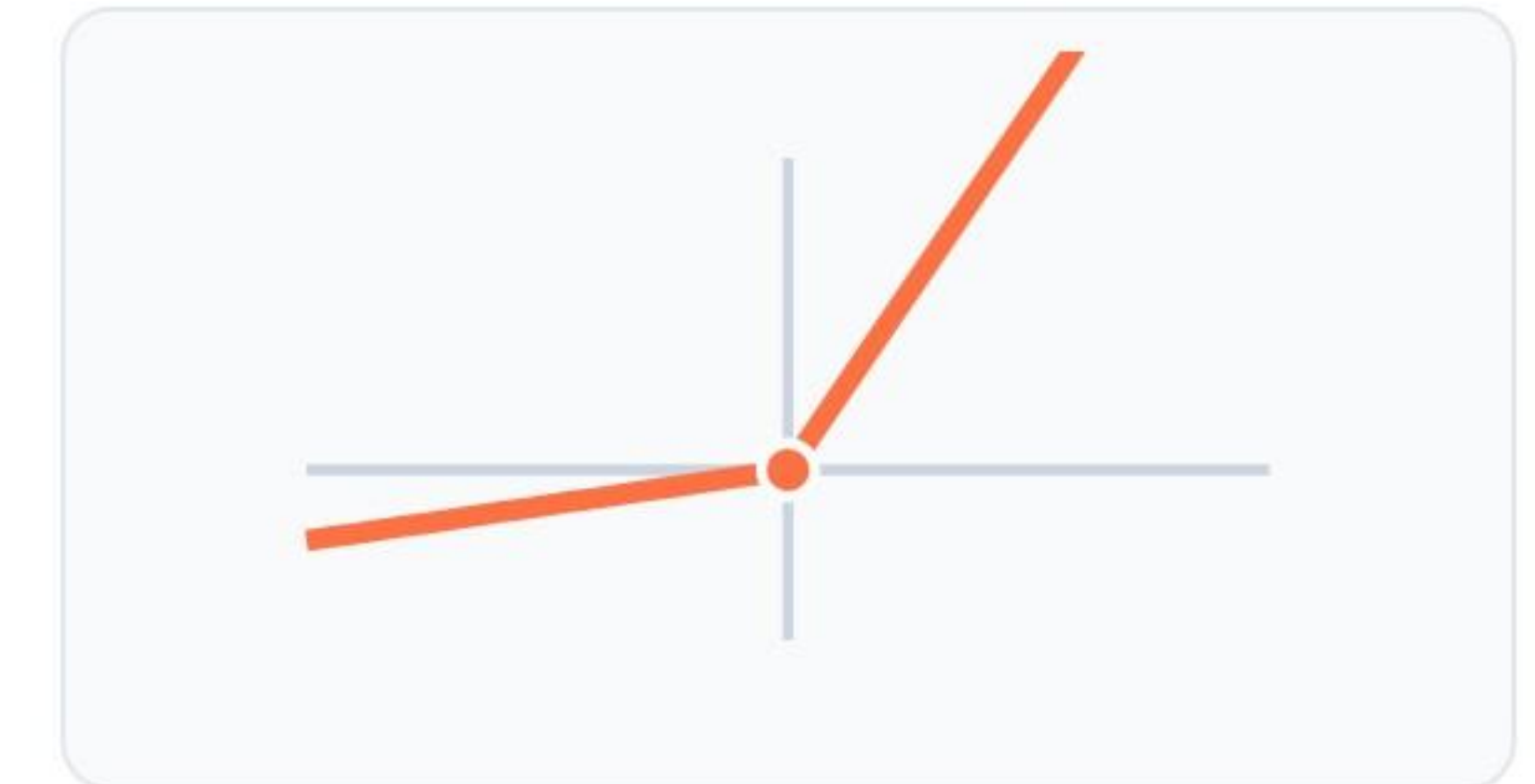
PREVINE NEURÔNIO MORTO

Intervalo: $(-\infty, +\infty)$

Leaky ReLU

$$f(z) = \max(0.1z, z)$$

Vazamento $\alpha = 0.1$ para que $z < 0$ continue aprendendo.

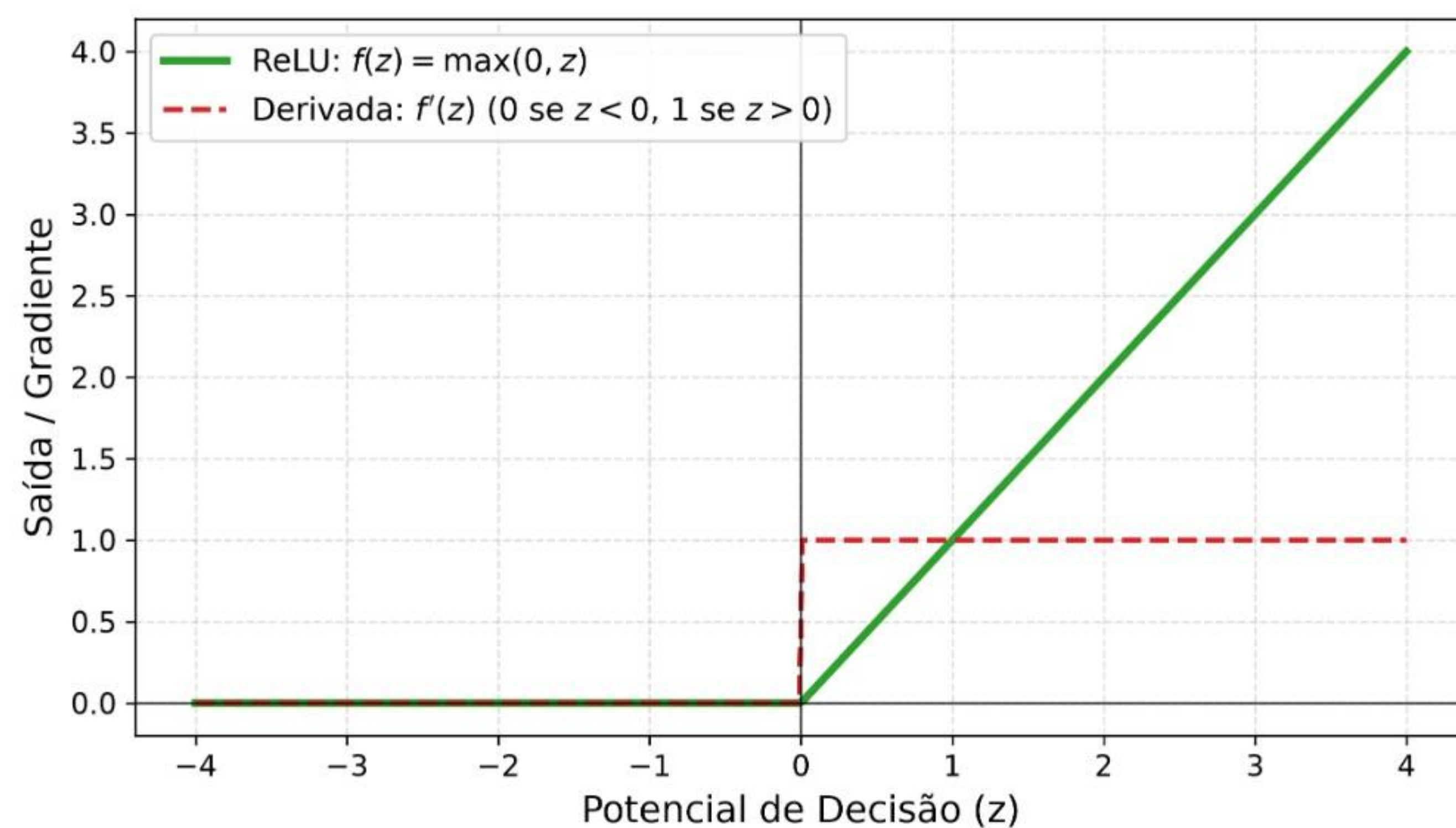


A Função ReLU (Rectified Linear Unit)

A ativação mais popular do Deep Learning moderno



Função de Ativação ReLU e sua Derivada (Sem Desvanecimento de Gradiente)



- ▶ Definição Matemática: $f(z) = \max(0, z)$.
- ▶ Se $z \leq 0$, a saída é 0. Se $z > 0$, a saída é o próprio z .
- ▶ Derivada $f'(z)$: É igual a 0 para $z < 0$, e igual a 1.0 para $z > 0$.
- ▶ Gradiente constante igual a 1.0 evita o desvanecimento do gradiente.

As 3 Vantagens Cruciais da ReLU

Por que a ReLU desbancou a Sigmoid nas camadas ocultas?



1. EFICIÊNCIA

Simplicidade Computacional

Exige apenas comparar $\text{if } (z > 0)$, economizando bilhões de operações exponenciais pesadas em GPUs.

2. APRENDIZADO

Gradiente Constante (1.0)

Com derivada igual a 1.0 para $z > 0$, o sinal do erro flui sem atenuação no backpropagation, combatendo o Vanishing Gradient.

3. REPRESENTAÇÃO

Esparsidade de Ativação

Zera neurônios inativos ($z \leq 0$), fazendo com que a rede ative apenas as características realmente relevantes.

Neurônio Morto & Leaky ReLU

Superando a limitação da ReLU tradicional



PROBLEMA: DYING RELU

O Neurônio Morto

- ▶ Se z for consistentemente negativo ($z < 0$), a saída será 0.
- ▶ O gradiente $f'(z)$ também torna-se 0.
- ▶ Com gradiente zero, os pesos NUNCA mais se atualizam.
- ▶ O neurônio fica "morto" permanentemente.

SOLUÇÃO: LEAKY RELU

Pequeno Vazamento ($\alpha = 0.1$)

- ▶ Formula: $f(z) = \max(0.1z, z)$.
- ▶ Para valores negativos, mantêm uma pequena inclinação (0.1).
- ▶ O gradiente para $z < 0$ passa a ser 0.1 em vez de zero.
- ▶ Permite que o neurônio continue aprendendo e se recupere.

Laboratório Interativo de Ativações

Explore o comportamento e os gradientes de cada função em tempo real



Sigmoid

Tanh

ReLU

Leaky

Entrada (z):

1.50

Saída $f(z)$:
0.8176

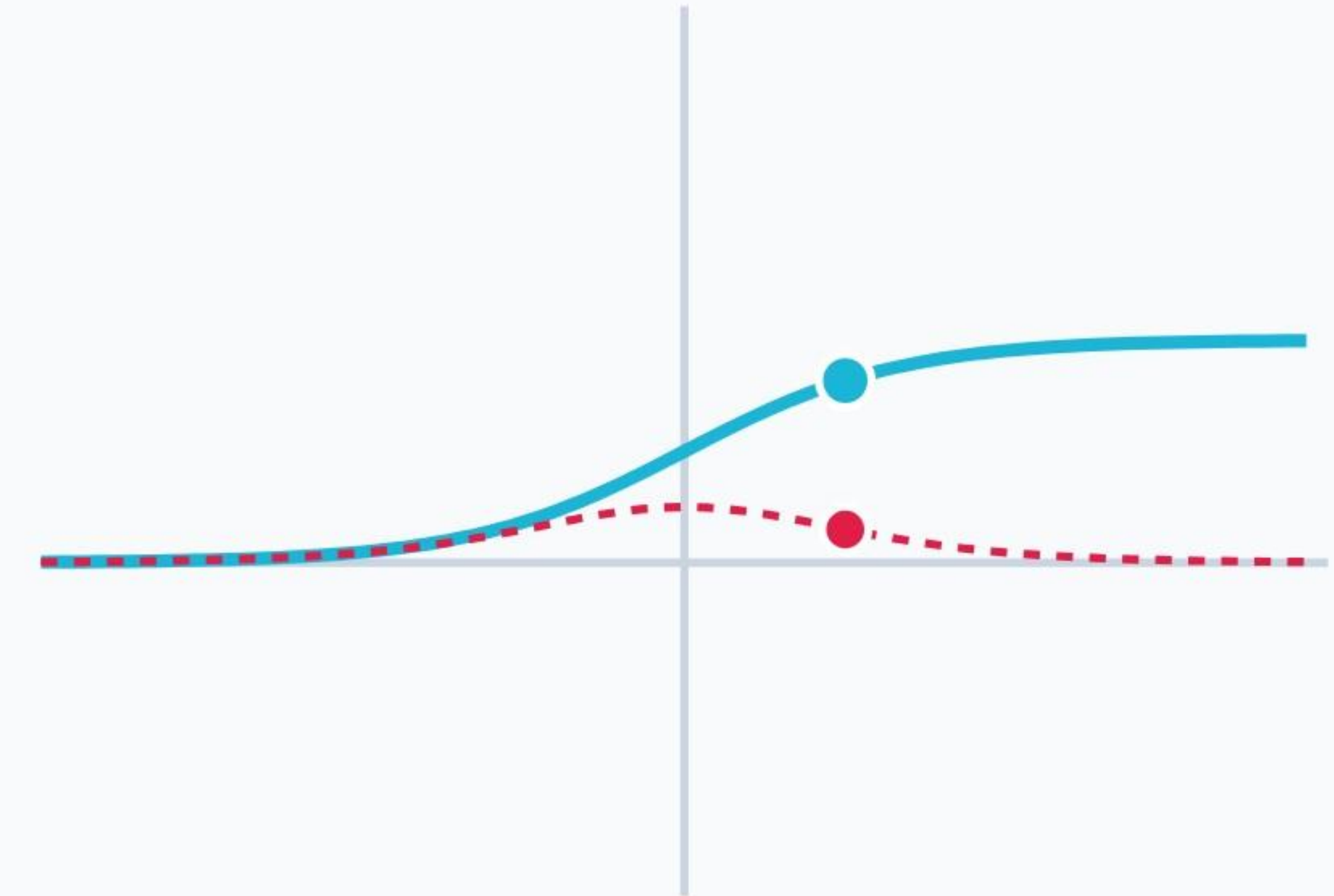
Gradiente $f'(z)$:
0.1491

Sigmoid (Intervalo: [0, 1])

Achata qualquer entrada para [0, 1]. Excelente para probabilidades no final de classificações binárias.

— $f(z)$

- - Derivada $f'(z)$



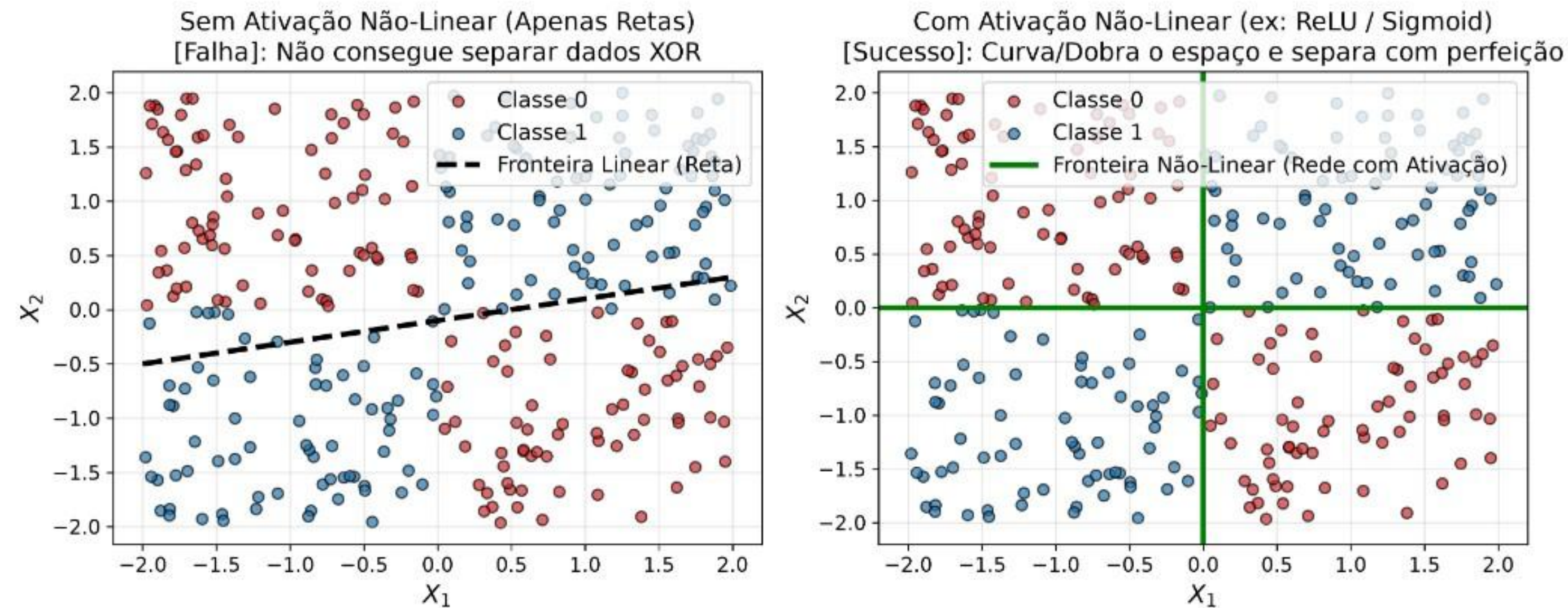
$$\sigma(z) = 1 / (1 + e^{-z})$$

O Limite do Perceptron: O Problema do XOR

Por que um único neurônio não resolve problemas não-lineares



- ▶ No gráfico da esquerda: Um Perceptron linear falha ao tentar separar o XOR com uma reta.
- ▶ Classes cruzadas em quadrantes não aceitam nenhuma linha reta sem cometer erros.
- ▶ Minsky & Papert (1969) provaram essa limitação, iniciando o primeiro "Inverno da IA".
- ▶ Solução: Adicionar camadas ocultas com funções de ativação não-lineares!



Curvando o Espaço: Solução Não-Linear

Veja como a não-linearidade curva o espaço para resolver o XOR



Tabela Verdade da Função XOR

Entrada x_1	Entrada x_2	Saída XOR (y)
0	0	0
0	1	1
1	0	1
1	1	0

● Perceptron Único (Linear)

● MLP Não-Linear (Camada Oculta)

✗ FALHA DE SEPARAÇÃO

Nenhuma reta no espaço 2D consegue separar os pontos verdes dos vermelhos simultaneamente! A acurácia máxima de um neurônio único é de apenas 50%.

Fronteira Linear (Reta Rígida)



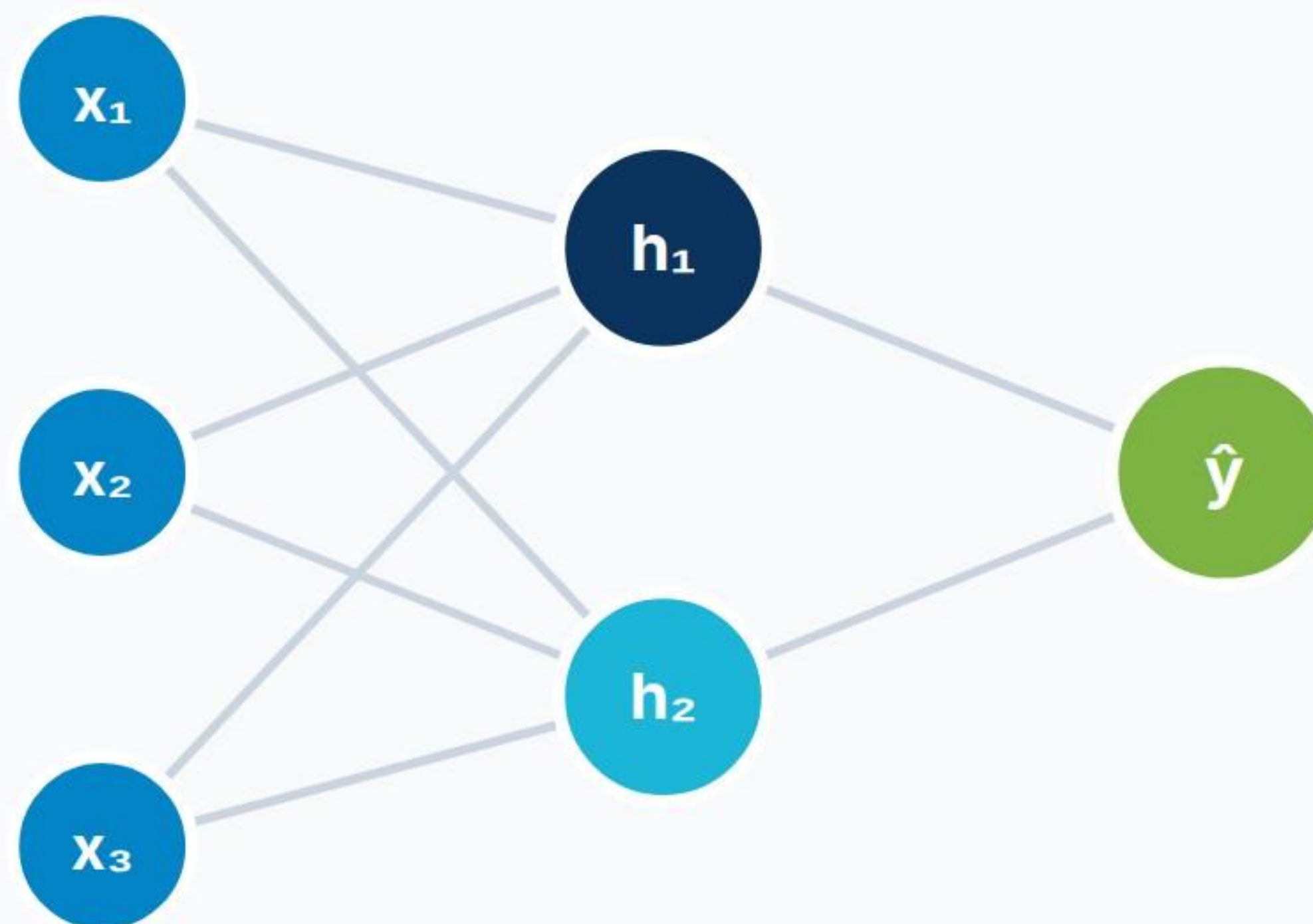
Redes Multi-Camadas (MLP)

Multi-Layer Perceptron: Combinando neurônios em camadas



Arquitetura Multi-Layer Perceptron (MLP)

⚡ Simular Propagação (Forward Pass)



Entrada (3 Neurônios)

Oculto (2 Neurônios)

Saída (1 Neurônio)

Camada Oculta

Neurônio Oculto h_1

Extrai um detector de características não-lineares a partir das combinações de entrada.

$$h_1 = \sigma(w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1)$$

$$h_1 = \sigma(w_{11}x_1 + w_{21}x_2 + b_{h1}), \quad \hat{y} = \sigma(v_1h_1 + v_2h_2 + b_o)$$

Camada Oculta (h_1, h_2)

Cada neurônio oculto processa as mesmas entradas x , aprendendo sub-fronteiras de decisão independentes.

Camada de Saída ($\hat{y}$)

Trata as saídas h_1 e h_2 como novas entradas e combina-as para gerar a decisão final.

PASSO 1

Abrir a Planilha

Abram o arquivo `simulacao_perceptron.xlsx` na pasta da aula 1.

PASSO 2

Alterar os Pesos

Modifiquem manualmente os valores de W_1 , W_2 e Bias nas células destacadas.

PASSO 3

Minimizar a Perda

Observem a Loss (Erro) despencar à medida que a reta se ajusta aos 20 atletas.

Transição para o PyTorch

Do cálculo manual para o framework de Deep Learning



PILAR 1

Tensores (`torch.Tensor`)

Arrays multidimensionais otimizados para aceleração paralela em GPUs.

PILAR 2

Autograd (`loss.backward()`)

Diferenciação automática que calcula todas as derivadas parciais instantaneamente.

PILAR 3

Otimizadores (`torch.optim`)

Algoritmos como SGD e Adam que atualizam os pesos automaticamente sem ajuste manual.

Quiz de Fixação & Próximos Passos

Testem seus conhecimentos antes de abrir o Jupyter Notebook!



Teste Rápido de Fixação de Conceitos

1. Qual é a principal função do Viés (Bias - b) no Perceptron?

Aumentar o número de parâmetros da GPU sem utilidade prática.

Deslocar a reta/plano de decisão para que ela não seja forçada a passar pela origem (0,0).

Garantir que a saída do neurônio seja sempre um número inteiro entre 0 e 100.

2. Por que a ReLU superou a Sigmoid nas camadas ocultas de redes profundas?

Sua derivada é constante e igual a 1 para entradas positivas, combatendo o desvanecimento do gradiente.

A ReLU produz probabilidades entre 0 e 1 perfeitamente calibradas.

A ReLU é uma função complexa que exige cálculo exponencial pesado.

3. Por que um Perceptron único não consegue resolver o problema do XOR?

Porque o XOR exige mais de 1 milhão de amostras para treinar.

Porque os dados do XOR são linearmente não-separáveis por uma única reta.

Porque a função Sigmoid não aceita valores negativos.

Verificar Respostas